

TD7 – Analyse des fraudes

Le fichier *creditcard.csv* est un jeu réel de transaction par carte de crédits effectué par des européens sur une période de deux jours. Les variables *V1* à *V24* sont anonymisées (confidentialité) sans signification directe résultant d'une analyse en composante principale. *Time* correspond le temps écoulé (en secondes) depuis la première transaction. *Amount* correspond au montant de la transaction en euros. *Class* est la variable à prédire (1 fraude, 0 normale).

Exercice 1 : Analyse des données

- Télécharger le fichier *creditcard.csv* quelle est la proportion des fraudes.
- Affichez la répartition des fraudes et des transactions normales au cours du temps. Y-a-t-il des instants de la journée plus propices aux fraudes.
- Comment se répartissent les transactions en fonction des sommes pour les transactions normales et pour les fraudes. Pour cela vous utiliserez la fonction *cut* de *pandas* en fonction d'un découpage en bac *bins=[-1, 100, 500, 1000, 2000, feature.max()]*. Que constatez-vous en particulier pour les transactions importantes (supérieures à 2500euros).
- Consulter la moyenne et l'écart type des variables *V*, que constatez-vous. Pensez-vous qu'il soit nécessaire de les normaliser.
- Pourquoi faut-il normaliser les features *Time* et *Amount*. Pensez-vous qu'il y ait des outliers ou des valeurs extrêmes.
- *RobustScaler* est une normalisation de *sklearn* qui centre les données sur la médiane alors que *StandardScaler* centre les données sur la moyenne. Pourquoi dans notre cas est-il plus raisonnable d'utiliser *RobustScaler*.
- Normalisez ces deux features en utilisant *RobustScaler*.
- Affichez les corrélations entre les différentes features et la feature *Class*. Quelle sont les variables les plus liées avec la target.
- Affichez des *boxplots* pour les 4 features les plus liées négativement à la target et pour les 4 features les plus liées positivement en fonction de la target. Que constatez-vous, et comment peut-on interpréter ces résultats.
- Affichez maintenant avec *scatterplot* la répartition des transactions par rapport aux deux features les plus corrélées avec la target.

Exercice 2 : Clustering sur les toutes les données

- Découper le data en données d'entraînement et données de test avec des taux de 66/33.
- Vérifiez que les pourcentages de fraudes sont équivalents dans le train et dans et dans le test. Pourquoi cette proportion doit-elle être respectée.
- Utiliser un **RandomForest** avec 500 estimateurs et une profondeur de 10. Pensez-vous que le score soit utilisable en l'état. Vous le comparerez avec le taux de fraudes mal classées.
- On souhaite utiliser un **IsolationForest** avec un taux de contamination de correspondant à la proportion du nombre de fraudes.
- Grâce à la fonction `apply` des `dataframe` transformer les résultats obtenus. Les valeurs -1 qui correspondent à de erreurs doivent être remplacés par des 1 et les 1 par de 0.
- Réalisez les mêmes apprentissages (**RandomForest**, **IsolationForest**) en utilisant que les 6 *features* les plus corrélées avec la *target*. Que constatez-vous.

Exercice 3 : Équilibrage des données

- On souhaite créer un jeu de données qui contient autant de fraudes que de transactions normales. Utiliser la fonction `sample` des `dataframe` pour sélectionner un échantillon aléatoire d'éléments.
- Créez un jeu de données *X* contenant autant de transactions normales que de transactions frauduleuses. Attention les fraudes et les transactions normales risquent de ne pas être mélangées. Pour cela utiliser la fonction `sample` avec l'option `frac=1` pour sélectionner 100% des données (qui seront donc mélangées).
- Pensez également à réindexer le jeu de données.
- Affichez les corrélations entre les *features* et la *target*. Que constatez-vous, essayez d'expliquer les différences avec celles du jeu complet.
- Affichez pour les dix variables les plus corrélées à la *target* et les variables *time* et *amount* (normalisées) les corrélations entre ces variables. Que peut-on conclure de ces niveaux de corrélations, en particulier sur les informations qu'elles fournissent pour modèles d'apprentissage.
- Affichez des `boxplot` pour les pour les 4 *features* les plus liées négativement à la *target* et pour les 4 *features* les plus liées positivement en fonction de la *target*. Que peut-on voir sur ces affichages et que peut-on en conclure sur les transactions normales et sur les fraudes.

- Affichez des **hisplot** dans les mêmes conditions. Les résultats confirment-ils les affichages des **boxplot**. Pensez-vous au regard de courbes qu'il sera possible d'identifier les fraudes. Y aura-t-il des fraudes difficiles à identifier.
- On souhaite voir si l'on peut visualiser les distributions des deux classes. **TSNE** (t-distributed Stochastic Neighbor Embedding) est une méthode de réduction de dimension non linéaire pour la visualisation. Utiliser cette méthode avec 2 composants pour entraîner et transformer (`fit_transform`) les données.
- Affichez les résultats avec un **scatterplot** sur les deux composantes en fonction de la classe. Que constatez-vous.
- Affichez via la fonction **describe()** comment se répartissent les transactions en fonction de leur classe. Commentez les résultats.
- Quel est la meilleure profondeur d'arbre pour les **RandomForest**, vous testerez des profondeurs de 1 à 20.
- Affichez les résultats et la matrice de confusion pour le meilleur modèle.

Informations :

- Le fonction `pd.cut(feature, bins=[v1, v2, v3 ... vn])` permet de découper la feature en bacs allant de `[v1, v2]`, `[v2, v3]`, ... `[vn-1, vn]`.